

ARAMS Hospital — TurtleBot3 Robotic Nurse

Object-Tag Mapping in a Simulated Hospital

**im Modul ARAMS – Advanced Robotics and Autonomous Mobile
Systems**

Gizem Bükülmez (3824106)

Eduardo Alvarado (3535659)

SS 2026

ARAMS Hospital — TurtleBot3 Robotic Nurse

ARAMS Projesi 2026 · FH Aachen

1 Task and overall approach

The robot in the project should act like a **robot nurse**: it should start from the starting point, read the active **AprilTag** on the panel to determine which patient pressed the call button, navigate to the correct room while avoiding a nurse walking down the hallway, and **classify the patient's states** as lying in bed (asleep), standing, or lying on the floor (unconscious). This entire process runs automatically there are no manual steps after the start commands. This is precisely what earns the highest score in the evaluation: full autonomy.

We built the solution on top of the basic loop that every autonomous robot runs: **perceive → decide → act**. The most important architectural decision here was: which parts will we write ourselves, and which parts will we use off-the-shelf? Path planning, obstacle avoidance, and costmaps are solved problems; instead of writing them from scratch, we use ROS 2's Nav2 stack. Similarly, we handle tag detection using the `apriltag_ros` package. The code we wrote ourselves consists of the mission logic that executes these off-the-shelf components in the correct order and the perception layer that converts the image from the camera into a decision. All of our work is consolidated into a single package: `arams_mission` (message definitions are in `arams_mission_msgs`).

2 System architecture

The task is divided into small, single-purpose nodes. The nodes communicate with each other via latched (persistent) topics; this ensures that no matter in what order a node starts, it can still catch up on information published before it. The table below summarizes the responsibilities of each component.

Phases	Node / module	Responsibility
1 — Perceive	<code>tag_reader_node</code> + <code>PinboardTagReader</code>	Read the pinboard AprilTag; lock ID after 5 consistent frames; publish it once as the mission target.
2 — Act	<code>pinboard_approach_node</code>	Drive to a fixed pose so the pinboard tag enters the camera FOV; gate tag reading on confirmed arrival.
2 — Act	<code>room_navigator_node</code> + <code>RoomNavigator</code>	Open the matching door, then send Nav2 to the room approach pose; classify the terminal outcome.
Ruese	Nav2 stack	Global/local planning,, costmaps, LIDAR obstacle avoidance consumed,not rebuilt.
5 — Perceiive	<code>patient_state_node</code> + <code>PatientDetector</code>	Run the trained YOLO model on the camera feed; <code>in_bed</code> / <code>on_floor</code> / <code>standing</code> .

There is a deliberate design principle that runs through every node: **keeping the application logic separate from the ROS layer**. Each node consists of a thin wrapper that manages subscriptions, parameters, and action clients. The actual decision-making, however, resides in pure Python classes (`PinboardTagReader`, `RoomNavigator`,

`PatientDetector`). This offers a major practical benefit: we can validate the decision logic through unit tests without needing a running ROS graph or simulator. This choice directly improves both the testability and readability of the code.

3 Mission flow

3.1 Reading the pinboard tag

The pinboard may not always be visible from the robot's starting pose. Therefore, the `pinboard_approach_node` is triggered first and sends a single `NavigateToPose` target to a fixed "tracking pose" where the pinboard is clearly visible to the camera. When Nav2 reports this target as `SUCCEEDED` and only then the node emits a persistent `/pinboard_read` signal. The `tag_reader_node` intentionally defers its subscription to `/detections` until this signal arrives. Why? Because the tags on the doors of the three rooms also use the same tag family and the same 1–3 ID range. If the robot attempts to read tags before it reaches the pinboard, it might accidentally lock onto a door tag it sees from a distance. Making arrival a prerequisite prevents this error from the start.

Even a single frame can result in a misread (motion blur, partial view). For this reason, the `PinboardTagReader` locks onto an ID only if it appears as the sole valid ID in five consecutive frames; the presence of a different ID or an empty frame resets the counter. Once locked, the ID is broadcast once and permanently as a `MissionTarget` message. We set the tracking distance to ~2.2 m: the camera is ~0.09 m above the ground and level, while the tag is at ~0.8 m; at closer distances, the top edge of the tag extended beyond the camera's vertical field of view.

3.2 Opening the door and navigating

`room_navigator_node` retrieves the permanently published target, opens the door by calling the `/open_door_N` service for the relevant room, and **only then** sends the Nav2 target for that room's approach pose. **The order is important:** a door mapped as closed remains a permanent obstacle in Nav2's static costmap layer—even if it is later actually opened. Therefore, it is mandatory to open the door before navigation.

The room and tracking positions were not manually adjusted; they were recorded in real time on the map (`tf2_echo map base_link`) and written to the `'config/rooms.yaml'` file. There are no hard-coded coordinates in the code—if a position changes, only the YAML is updated. Avoiding the walking nurse is handled entirely by Nav2's **local costmap and LIDAR**; we didn't write this behavior we simply utilized it.

The navigation result is classified into an explicit `NavigationOutcome` enum (`SUCCEEDED` / `ABORTED` / `CANCELED` / `REJECTED` / `UNKNOWN_ROOM`); this ensures that failures are logged individually rather than being silently ignored.

3.3 Detecting the patient's state

`patient_state_node` runs a YOLO model fine-tuned on three patient states on the camera stream and publishes the class with the highest confidence score as a `PatientState` message. The model is trained offline, outside of ROS (`training/train_yolo.py`). We created the training dataset using our custom `collect_images`, collecting several hundred frames by manually maneuvering the robot around each patient.

4 Autonomy, robustness and clean execution

- 1 **Fully autonomy:** After two startup commands, the robot completes the entire task approaching the board, reading, opening the door, navigating, and detecting without any manual intervention.
- 2 **Order-independent startup:** Latched (transient-local) QoS is used in each task topics; thus, a node does not lose its state even if it starts after the information has been broadcast.
- 3 **Fail-safe gating:** Label reading never begins from an unverified position; a failed approach cleanly suspends the task rather than proceeding with incorrect data.
- 4 **Clean execution:** Planning and avoidance are delegated to Nav2 with speed and Radius limits specific to the TurtleBot3 Burger; this prevents erratic movement and collisions with the nurse.
- 5 **Repeatability testing:** An optional CSV logger and summarizer counts navigation results across multiple trials and measures gate-approach reliability against a success threshold.

5 Code quality and testing

The codebase follows the clean coding principles referenced in the project guide (Uncle Bob — Clean Code). **DRY** (Don't Repeat Yourself): Two navigation nodes share the `interpret_result` and `yaw_to_quaternion_z_w` functions. **KISS** (Keep It Simple): The fine-wrapper node design is a direct result of this. Naming makes the intent clear from start to finish such as `required_consistent_detections` instead of a constant.

The decision logic is covered by a suite of unit tests that run without a simulator: `test_pinboard_tag_reader`, `test_room_navigator`, `test_patient_detector`, `test_trial_summary` and others. Configuration (valid IDs, poses, thresholds, model weight path) is fully externalized to YAML; thus, no code changes are required to modify behavior.

6 How to build and run

Full instructions are in the repository README. In short, after building with colcon and sourcing the workspace:

```
ros2 launch arams_mission navigation_launch.yaml
ros2 launch arams_mission tag_reader_launch.yaml
ros2 launch webots_ros2_turtlebot arams_challenge_launch.py
```